

Bare-Metal OS for a Raspberry Pi4

Project Report for the Operating Systems Course
University of Basel, Spring Semester 2026

Authors: Andrea Seehuber
 Luca Fässler

Contents

1	Introduction	1
2	Hardware and Software Environment	1
3	System Architecture	1
4	Core Components	2
4.1	Bootstrapping and Runtime Setup	2
4.2	Task System and Cooperative Scheduler	2
4.3	Synchronization and Shared Hardware Access	3
4.4	Device Drivers and I/O Layer	4
5	Evaluation and Limitations	4

1 Introduction

The aim of this project was to implement a small bare-metal operating system for the Raspberry Pi4. The system is loaded directly as a kernel image and does not rely on Linux, a standard C runtime, processes, filesystems, or user-space services. Its main purpose is to demonstrate how core operating system mechanisms can be built from scratch on real hardware.

The project focusses on bootstrapping, kernel initialization, cooperative multitasking, context switching, synchronization, hardware drivers, and interactive system control. The resulting system is not intended to be a general-purpose operating system. Instead, it is a compact embedded kernel that can schedule multiple kernel tasks, communicate over UART, render text output to HDMI, access external hardware over I2C, and provide runtime diagnostics.

2 Hardware and Software Environment

The target platform is a Raspberry Pi4 Model B. The board is based on a Broadcom BCM2711 system-on-chip with a 64-bit ARM Cortex-A72 CPU. For this project, the Raspberry Pi is used without a conventional operating system. The firmware loads the custom kernel image directly from the boot partition, after which all further initialization is performed by the project code itself.

As an extension module, the system uses the Raspberry Pi Sense HAT. This add-on board is connected through the 40-pin GPIO header and provides an 8x8 RGB LED matrix, a five-way joystick, and several environmental and motion sensors. In this project, the relevant Sense HAT components are the joystick, the LED matrix, and the sensors for pressure, humidity, and temperature. These devices are accessed through the Raspberry Pi's I2C controller.

UART is used as the primary debugging and control interface. It allows the kernel to print boot messages and expose an interactive shell even when HDMI output is unavailable or currently owned by a specific task. HDMI is used as the main visual output device. The kernel obtains a framebuffer through the Raspberry Pi firmware mailbox interface and implements its own text rendering on top of it.

The software is written mainly in C, with architecture-specific AArch64 assembly for early bootstrapping and context switching. The project is built using a Makefile. The build system uses an AArch64 bare-metal cross toolchain and compiles the code in freestanding mode, without the standard C library or startup files. The linker script defines the memory layout of the kernel image, and the final binary is converted into a Raspberry Pi compatible `kernel8.img`.

The implementation was developed with reference to the tutorial series **Writing a Bare-Metal Operating System for Raspberry Pi4**. The tutorial provided the initial orientation for the bare-metal boot process, cross-compilation setup, linker configuration, and the general structure of an AArch64 Raspberry Pi kernel[1]. For hardware-specific register-level programming, the project also relies on the official **BCM2711 ARM Peripherals** documentation published by Raspberry Pi Ltd[2].

3 System Architecture

The system follows a small monolithic kernel architecture. All kernel subsystems, drivers, synchronization primitives, and built-in tasks run in the same address space. There is no separation between user mode and kernel mode. This keeps the implementation compact and makes direct hardware access possible, but it also means that memory protection and process isolation are not provided.

The boot flow starts in AArch64 assembly. The startup code prepares the execution environment, sets up the initial stack, clears uninitialized global data, and transfers control to the C kernel entry point. The C kernel then initializes low-level drivers, the timer, interrupt handling, memory management structures, synchronization primitives, the task system, and finally the scheduler.

The kernel is organized around three layers:

- The lowest layer contains hardware-specific drivers such as UART, timer, I2C, HDMI frame-buffer access, and Sense HAT device support.
- The middle layer contains kernel mechanisms such as task management, scheduling, mutexes, logging, tracing, and diagnostic accounting.
- The upper layer consists of built-in kernel tasks such as the shell, joystick menu, environmental monitoring, LED animations, games, and dashboards.

This structure keeps hardware access separated from application-like kernel tasks. Tasks do not need to manipulate low-level hardware registers directly. Instead, they use driver and kernel interfaces such as console output, HDMI pane rendering, sensor access, task control, or LED matrix rendering.

4 Core Components

4.1 Bootstrapping and Runtime Setup

Because the system runs bare-metal, there is no operating system loader or standard runtime that prepares the execution environment. The first project code executed after firmware loading is therefore written in assembly. This code performs the minimum setup required before C code can run safely.

The startup code establishes an initial stack and clears the `.bss` section. This step is essential because global kernel structures such as task tables, buffers, mutexes, counters, and driver state depend on zero initialization. In a hosted environment this would be handled by the runtime, but in this system it must be done manually.

After the low-level initialization, the assembly code branches to the C kernel entry point. From this point on, initialization proceeds in a structured order. UART is initialized early so that debugging output is available. The kernel then initializes interrupt handling, the timer, memory management, synchronization primitives, hardware drivers, and the task subsystem. Once the initial kernel tasks have been created, the scheduler is started.

The project also uses assembly for context switching. While the scheduling decision is made in C, switching from one task to another requires saving and restoring CPU register state and replacing the current stack pointer. This cannot be implemented safely in ordinary C code because the stack itself is part of the task context being switched.

4.2 Task System and Cooperative Scheduler

Each **task** is represented by a task control block (TCB). The TCB stores the task ID, current state, stack pointer, entry function, name, runtime counters, wakeup time, and private stack. The task table is statically allocated, which keeps memory usage deterministic and avoids depending on dynamic allocation for basic scheduling.

When a task is created, the kernel prepares its stack manually. A new task has never run before,

but the context switch routine expects a stack that looks like a previously saved execution context. Therefore, task creation builds an artificial initial stack frame. The saved return address points to a bootstrap function. When the scheduler selects the task for the first time, the low-level context switch restores this artificial context and returns into the bootstrap function.

The bootstrap function calls the task's entry function. If the entry function returns, the task is marked for termination and control is returned to the scheduler. This design allows the same context switch mechanism to be used both for starting new tasks and resuming existing tasks.

The **scheduler** is cooperative. A task continues to run until it explicitly yields, sleeps, blocks, or terminates. The scheduler uses task states such as ready, running, sleeping, blocked, dying, and unused:

- **Ready** tasks can be selected for execution.
- **Sleeping** tasks store a wakeup tick and are made ready again by timer-driven logic once the wakeup time has passed.
- **Blocked** tasks wait for an event or synchronization primitive.
- **Dying** tasks are cleaned up at a scheduler-safe point.

The scheduling policy is simple round-robin. Starting after the currently running task, the scheduler scans the task table for the next ready non-idle task. If no regular task is ready, the **idle task** is selected. The idle task waits for events and yields again. This keeps the scheduler small and predictable, but it also means that task responsiveness depends on tasks yielding regularly.

4.3 Synchronization and Shared Hardware Access

Even with cooperative scheduling, synchronization is required because multiple tasks can access shared kernel objects and because interrupts may update state asynchronously. The kernel therefore uses short critical sections where interrupts are disabled while sensitive data structures are modified. This is used for operations such as changing task state, waking tasks, updating scheduler data, or manipulating synchronization primitives.

For longer shared-resource protection, the kernel provides **mutexes**. A mutex records whether it is locked and which task owns it. If another task tries to acquire the mutex while it is already locked, the task is put into a wait queue, marked as blocked, and the scheduler selects another runnable task. When the mutex is released, one waiting task is woken up and can continue later.

The I2C bus is protected by such a mutex. This is necessary because the Sense HAT exposes several devices through the same bus. Sensor reads, joystick access, and LED matrix updates must not interleave their I2C transactions. The project therefore provides a global I2C bus lock. Every higher-level device driver acquires this lock before starting a transaction and releases it afterwards. This serializes bus access and prevents corrupted communication.

A similar ownership principle is used for display output. HDMI output is divided into panes, and tasks can acquire exclusive ownership of a pane before rendering application-specific output. The LED matrix is also treated as a shared hardware resource. Instead of allowing arbitrary concurrent writes, LED output is routed through controlled rendering code so that hardware access remains centralized.

4.4 Device Drivers and I/O Layer

The **UART driver** provides the lowest-level text output and input path. It is initialized early and remains the most reliable interface during development. The shell is built on top of UART input and allows direct control over tasks, logs, traces, and diagnostics.

The **timer driver** provides the time base for sleeping tasks, uptime measurement, scheduling counters, and periodic diagnostics. It is also used by tasks that update displays or sensors reading at regular intervals.

The **I2C layer** provides register-level communication with Sense HAT devices. Higher-level drivers use it to read environmental sensors, poll the joystick, and update the LED matrix. Access to the I2C bus is synchronized through the global bus lock, so the individual device drivers do not corrupt each other's transactions.

HDMI output is implemented using a framebuffer allocated through the Raspberry Pi firmware mailbox interface. The kernel implements its own rendering on top of this framebuffer. To support multiple output producers, the framebuffer is divided into logical panes. The main pane is used by applications and dashboards, while the menu pane is used for joystick-controlled system interaction. This allows the system to remain usable from the Sense HAT joystick without overwriting the main application display.

5 Evaluation and Limitations

The implementation demonstrates that the kernel can boot directly on Raspberry Pi4 hardware, initialize its own runtime environment, create and switch between multiple tasks, synchronize access to shared hardware, and expose both serial and visual control interfaces. The built-in tasks validate different parts of the system: worker tasks exercise the scheduler, sensor tasks validate I2C access, LED tasks validate display arbitration, and HDMI dashboards validate framebuffer rendering and pane ownership.

The main limitation is the cooperative scheduler. Since tasks are not preempted by a timer interrupt, every task must voluntarily yield, sleep, or block. A task that performs a long-running computation without yielding can reduce responsiveness. The system also does not implement user/kernel separation, virtual memory, process isolation, a filesystem, or memory protection. These features are outside the scope of the project, but they define clear directions for future work.

References

- [1] Sypstraw. *Writing a "bare metal" operating system for Raspberry Pi 4*. Tutorial series for bare-metal Raspberry Pi 4 operating-system development. 2020. URL: <https://www.rpi4os.com/> (visited on 05/31/2026).
- [2] Raspberry Pi Ltd. *BCM2711 ARM Peripherals*. Official Raspberry Pi 4 BCM2711 peripheral documentation. Jan. 18, 2022. URL: <https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008248-DS-1-bcm2711-peripherals.pdf> (visited on 05/31/2026).